

READING SHEET

Next.js Foundations Project

Module 3 · Frontend: React & Next.js · Day 40 · Week 8

PROGRAM CodeOps · Full Stack	WORK TYPE Project + capstone migration	EST. TIME 4–6 hours
--	--	-------------------------------

Tip How to use this sheet

This is a working document rather than revision. Part 1 is the brief and the rubric. Part 2 is the four decisions to make on paper before writing a page. Part 3 is how to verify each of them mechanically. Part 4 walks the capstone migration. Keep it open beside the terminal, and run the verification commands as you reach them.

Part 1 · The Project Brief

Rebuild **Addis Eats** on Next.js: the same food-ordering application from Day 35, now with file-based routing, nested layouts, a deliberate rendering strategy per route, server components fetching their own data, and your own endpoints and actions behind it. Rebuilding something you have already written means every difference is visible — you are not learning the problem again, you are seeing what the framework changed.

ROUTE	RENDERING	WHAT IT DOES
/	Static	Landing page with today's specials
/menu	ISR	Fetches dishes, filter, category sidebar
/menu/[id]	Static via params	One dish, add to order
/cart	Client	Order lines and the running ETB total
/checkout	Dynamic	Server action, validated, guarded

Required features

FROM DAY	MUST APPEAR IN YOUR BUILD
36	Folder routing, a dynamic segment, loading and error files
37	Nested layouts, and a justified strategy for every route
38	Server components fetching data; "use client" on leaves only
39	One route handler, one server action, server-side validation
Earlier	The cart, the form and the guard — carried across and working

Carry the old work over. The validation rules, the accessible field markup and the cart logic are already written — reuse them. Rewriting them from memory is wasted time, not extra credit.

Definition of done

NOT DONE	DONE
It works in npm run dev	It works from npm run build and start
Every route says λ	Each marker matches your written intent
"use client" everywhere	A boundary you can defend line by line
The form posts and hopes	Status codes handled, errors shown
Validation only in the browser	The same schema enforced server-side
A secret in the bundle	Nothing sensitive readable by the client

Warning Build, do not dev

Every line above can be verified rather than argued about — by reading the build output, searching for a directive, or sending a request with curl. And a production build catches serialisation errors, boundary mistakes and missing environment variables that the dev server quietly tolerates.

How it is marked

CRITERION	WEIGHT
Routes, layouts and features working	35%
Rendering strategy, chosen and justified	20%
Server and client boundary placement	20%
Endpoint design, validation and security	15%
Documentation and walkthrough	10%

Forty per cent of the mark is for choices you can explain — why this route is static, why that component is a client component. Features alone will not carry a project this week, and STRATEGY.md and BOUNDARY.md are deliverables rather than optional extras.

Part 2 · Four Decisions, On Paper

Each of these is expensive to change once code depends on it. Settle all four before creating a single file.

1 · The folder tree

```
app/                                     app/ – write this out before you create anything
  layout.js page.js not-found.js actions.js
  menu/ layout.js page.js loading.js error.js
      [id]/page.js
  cart/page.js checkout/page.js
  api/dishes/route.js api/orders/route.js
```

Unlike Day 35, the folder structure is not a preference — it *is* the routing. Shared code sits outside: lib/ and components/ at the project root for anything used by more than one route, with only route-specific components inside app/.

2 · The rendering strategy

ASK OF EVERY ROUTE	AND YOU HAVE YOUR ANSWER
Who is asking?	Nobody in particular → it can be static
How stale may it be?	An hour → revalidate = 3600
Do I know the ids?	Yes → generateStaticParams
Does it read a cookie?	Yes → dynamic, and that is correct
Is it private to one tab?	Yes → render it on the client

Write the answer into STRATEGY.md as one line per route: `/menu` `ISR 3600` `dishes` `change a few times a day`. A strategy you cannot justify in a sentence is a strategy you guessed — and writing the sentence usually changes the answer for at least one route.

3 · The server-client boundary

BOUNDARY.md — one line per component		
<code>app/menu/page.js</code>	<code>server</code>	<code>awaits dishes</code>
<code>DishList, DishCard</code>	<code>server</code>	<code>markup only</code>
<code>CategoryBar</code>	<code>client</code>	<code>selection state</code>
<code>AddToCartButton</code>	<code>client</code>	<code>onClick, writes to store</code>
<code>Providers</code>	<code>client</code>	<code>context needs state</code>
<code>app/layout.js</code>	<code>server</code>	<code>passes children in</code>

Deciding this in advance stops the directive spreading upwards as you build. If your list has more than four or five client components, look again at what genuinely needs interactivity. Note which client components wrap server content — those must receive it as children, so build them that way from the start.

4 · How data moves

WHAT	HOW	WHY THAT ONE
Menu and dish data	Server component	Read to render — no endpoint needed
Placing an order	Server action	A write from your own UI
Order status polling	Client fetch	Repeated, triggered by the browser
A payment confirmation	Route handler	Called by something you do not control
Cart contents	Client store only	Never leaves the browser until checkout

Warning The endpoint you do not need

The most common mistake this week is building `/api/dishes` and then calling it from your own server component. The component can query the data directly — an endpoint only exists if something outside your pages needs it. Build the handler because the polling client requires it, not out of habit.

Part 3 · Build & Verify

#	BUILD IN THIS ORDER
1	Folder tree and layouts — every route reachable, even if empty
2	The menu as a server component awaiting real data
3	The dish route, with generateStaticParams and notFound()
4	Client leaves: category bar, cart button, providers
5	The orders endpoint and the placeOrder action, with validation
6	Strategies, loading and error files, then the documentation

Build everything as a server component and add "use client" only when something forces it — working server-first means the boundary ends up where you planned rather than wherever you first hit an error. Run `npm run build` at the end of each of the six steps.

Verifying the strategy

<pre> Route (app) Size First Load JS ├── ○ / 1.1 kB 88 kB ├── ○ /menu 2.6 kB 94 kB ├── ● /menu/[id] 1.4 kB 90 kB └── λ /checkout 3.2 kB 97 kB // ○ static ● prerendered from params λ dynamic </pre>		what you want to see
IF YOU SEE	GO AND LOOK FOR	
λ where you wanted ○	A cookie, header or searchParams read	
○ where you wanted λ	A missing no-store or force-dynamic	
First Load JS jumped	A "use client" that spread further than intended	

Open STRATEGY.md beside the build output and check them line by line. Any disagreement is either a bug or a decision you have not thought through.

Verifying the endpoints

<pre> curl -i localhost:3000/api/dishes/not-a-dish # expect: HTTP/1.1 404 and { "error": ... } curl -i -X POST localhost:3000/api/orders \ -H "Content-Type: application/json" \ -d '{"name":"Almaz","phone":"0912"}' # expect: HTTP/1.1 422 and fieldErrors.phone </pre>	curl — before any UI touches them
--	-----------------------------------

Three commands prove the contract in seconds. Then attack your own action: call `cancelOrder` from the browser console with an id that is not yours. If it succeeds, the check is in the wrong place.

The six failure checks

DO THIS	AND CHECK
Throttle to Slow 3G	loading.js appears; the shell is already there
Throw inside the menu page	error.js renders, header and cart survive
Open /menu/not-a-dish	not-found.js, not a crash
Post an invalid phone number	422, and the message beside the field
Disable JavaScript, submit	The server action still places the order
Search the bundle for a secret	Nothing sensitive is there

Run these on the production build, not the dev server. The last two are new this week — and if the order still arrives with scripting turned off, your form is genuinely progressive, which is worth demonstrating in the walkthrough.

Part 4 · Capstone Migration

#	STEP
1	Scaffold a new Next.js project beside the old one
2	Recreate the route map as folders with empty pages
3	Move the layout into app/layout.js, Providers into a client file
4	Move each screen across, deleting its fetching hook as you go
5	Add loading, error and not-found files per segment
6	Write STRATEGY.md and BOUNDARY.md for your own routes

Move one screen at a time and keep the application building — a migration attempted in a single commit is very hard to debug when it does not work. Most of your code is unchanged: components, styles and validation move across untouched. What changes is where they live and which of them fetch their own data.

What usually goes wrong

SYMPTOM	CAUSE
"needs useState" everywhere	A hook in a component left on the server
Bundle barely smaller	"use client" at the top of a page
Functions cannot be passed	A callback prop crossing the boundary
Hydration mismatch	A date or random value rendered in JSX
Every route marked dynamic	A cookie read in a shared helper
404 on a route that exists	A missing or non-default export

Every one of these was covered on Days 36 to 39 — when you hit one, the fix is in that day's reading sheet rather than in a search engine. Next.js error messages name the file and usually the offending import, and the first two lines almost always contain the answer.

Presenting the comparison

MINUTES	WHAT TO COVER
0–1	What it does, and which routes exist
1–3	Live walkthrough on the production build
3–5	Your strategy table, and one route you changed your mind about
5–7	Your boundary map, and the bundle before and after
7–8	One endpoint demonstrated with curl, and questions

The First Load JS figures from Day 35 and today are the clearest evidence of what server components changed — have both open and say what moved and why. Present from `npm run start`, not the dev server: it is faster, it is what you were assessed on, and it will not recompile mid-demonstration.

Assigned Reading

Required

1. [nextjs.org — Migrating from Vite](https://nextjs.org/docs/advanced-features/migrating-from-vite) What / why: the official step-by-step for exactly the move you are making today.
2. [nextjs.org — Project structure](https://nextjs.org/docs/advanced-features/project-structure) What / why: the recommended folder conventions and where shared code belongs.

Optional — go deeper

- [Production checklist](#) — what to verify before a build is considered ready.
- [TanStack Query — quick start](#) — preparation for tomorrow, when week 9 opens with client-side data fetching.

Exercises

The first three are planning. Do them before creating any files.

1. Write out the full app/ folder tree on paper, including the special files.
2. Write STRATEGY.md with one line of reasoning for every route.
3. Write BOUNDARY.md listing every component and which side it runs on.
4. Build steps 1 to 3, running `npm run build` after each.
5. Build steps 4 to 6, then compare the build output against STRATEGY.md.
6. Prove every endpoint with curl and attack your own action from the console.
7. Run the six failure checks on the production build, then migrate the capstone.

Deliverables · Two Repositories

Tip What you submit

Addis Eats complete on Next.js with STRATEGY.md and BOUNDARY.md, verified against a production build and curl. Plus your capstone migrated to Next.js with every route reachable and its own strategy and boundary documented. Both pushed to GitHub, both verified from a fresh clone.

Project requirements

- Five routes as folders, with layouts nested and all special files present.
- A rendering strategy per route matching the build output marker exactly.
- Server components fetching data, with "use client" only on interactive leaves.
- One route handler and one server action, both validating with the shared schema.
- Authorisation checked inside the action, not in the component rendering the button.
- STRATEGY.md, BOUNDARY.md and the build output in the README.

Capstone requirements

- Every route from your Day 35 route map reachable on Next.js.
- The layout moved to app/layout.js, with providers isolated in a client file.
- At least one screen converted to a server component with its fetching hook deleted.
- STRATEGY.md and BOUNDARY.md written for your own routes and components.

Check yourself

- Does every marker in the build output match what STRATEGY.md claims?
- How many client components are there, and can you defend each one?
- Which part of your application actually calls /api/dishes?
- What did First Load JS for /menu do between Day 35 and today?
- Does the checkout still work with JavaScript disabled?
- Can you find any secret in the JavaScript the browser downloaded?
- Does your capstone build and run from a fresh clone?